

Capítulo 4: The Mathematics of Biological Systems

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

3 de janeiro de 2026

Sumário

- 1 Introdução
- 2 Sistemas de Equações Diferenciais
- 3 Análise de Estabilidade
- 4 Bifurcações
- 5 Oscilações e Ciclos Limite
- 6 Análise de Sensibilidade
- 7 Exercícios
- 8 Referências

Visão Geral do Capítulo

Objetivos de Aprendizagem

- Dominar ferramentas matemáticas avançadas para sistemas biológicos
- Compreender sistemas de equações diferenciais ordinárias (EDOs)
- Realizar análise de estabilidade e linearização
- Estudar bifurcações e transições de comportamento
- Analisar oscilações e ciclos limite
- Aplicar análise de sensibilidade paramétrica

Por que Matemática Avançada?

A matemática é essencial para:

- Predizer comportamento dinâmico de sistemas

Matemática como Linguagem da Biologia

Definição: Modelagem Matemática

A modelagem matemática traduz processos biológicos em equações que capturam a essência dinâmica do sistema, permitindo simulação, análise e predição quantitativas.

Níveis de Complexidade:

- Modelos lineares (simples)
- Sistemas não-lineares (realistas)
- Modelos estocásticos (com ruído)
- Modelos espaciais (PDEs)

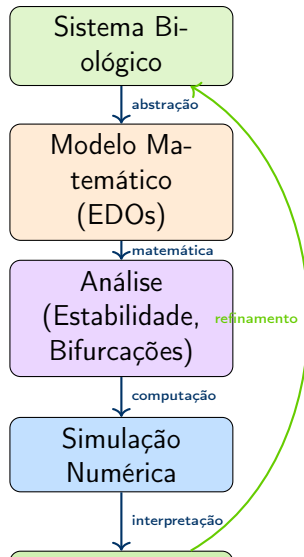
Ferramentas Principais:

- Equações diferenciais
- Análise de estabilidade
- Teoria de bifurcações
- Métodos numéricos

Importante!

Este capítulo foca em sistemas dinâmicos determinísticos descritos por EDOs

Visão Integrada: Do Modelo à Predição



Tópicos Avançados deste Capítulo

Parte I: Dinâmica

1. Sistemas de EDOs
2. Retratos de fase
3. Análise de estabilidade
4. Linearização e Jacobiano

Parte II: Transições

5. Bifurcações
6. Oscilações e ciclos limite

Parte III: Análise

7. Análise de sensibilidade
8. Exploração de espaço paramétrico

Aplicações

- Lotka-Volterra
- Van der Pol
- Modelos de regulação gênica
- Osciladores biológicos

Revisão: Equações Diferenciais Ordinárias

Definição: Equação Diferencial Ordinária (EDO)

Uma EDO relaciona uma função desconhecida $y(t)$ com suas derivadas:

$$\frac{dy}{dt} = f(y, t)$$

onde t é a variável independente (geralmente tempo) e y é a variável dependente.

EDO de 1ª Ordem

$$\frac{dx}{dt} = -kx$$

Solução: $x(t) = x_0 e^{-kt}$
(decaimento exponencial)

EDO de 2ª Ordem

$$\frac{d^2x}{dt^2} + \omega^2 x = 0$$

Solução: $x(t) = A \cos(\omega t) + B \sin(\omega t)$
(oscilador harmônico)

Sistemas de EDOs Não-Lineares

Forma Geral

Um sistema de n equações diferenciais de primeira ordem:

$$\frac{dx_1}{dt} = f_1(x_1, x_2, \dots, x_n, t)$$

$$\frac{dx_2}{dt} = f_2(x_1, x_2, \dots, x_n, t)$$

$$\vdots$$

$$\frac{dx_n}{dt} = f_n(x_1, x_2, \dots, x_n, t)$$

Forma Vetorial:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$$

onde $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ e $\mathbf{f} = (f_1, f_2, \dots, f_n)^T$

Campos Vetoriais e Retratos de Fase

Definição: Campo Vetorial

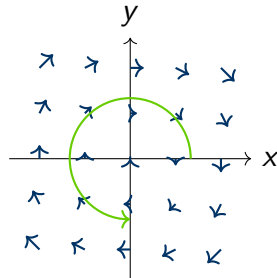
O lado direito do sistema $f(x)$ define um campo vetorial que associa a cada ponto do espaço de estados um vetor velocidade.

Exemplo Simples:

$$\begin{aligned}\frac{dx}{dt} &= y \\ \frac{dy}{dt} &= -x\end{aligned}$$

Interpretação:

- Em cada ponto (x, y) existe um vetor $(\dot{x}, \dot{y})$

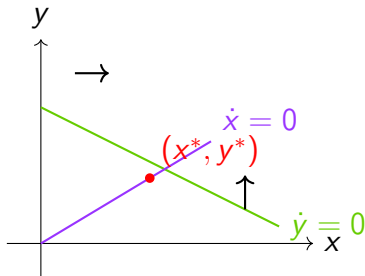


Nulclinas

Definição: Nulclina

Curva no espaço de fase onde uma das derivadas é zero:

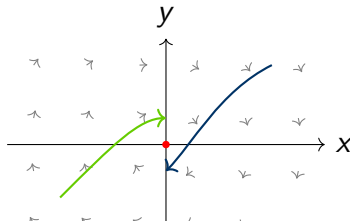
- **x-nulclina:** onde $\dot{x} = 0$ (movimento apenas vertical)
- **y-nulclina:** onde $\dot{y} = 0$ (movimento apenas horizontal)



Campos de Direção

Construindo Retratos de Fase

1. Calcular e plotar nulclinas ($\dot{x} = 0$ e $\dot{y} = 0$)
2. Identificar pontos de equilíbrio (interseções)
3. Desenhar campo de direções (vetores $(\dot{x}, \dot{y})$)
4. Integrar trajetórias numéricas a partir de condições iniciais
5. Analisar comportamento assintótico



Resolvendo EDOs com SciPy

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # Definir sistema de EDOs
6 def sistema(X, t, alpha, beta):
7     """Sistema de duas variaveis acopladas"""
8     x, y = X
9     dxdt = alpha*x - beta*x*y
10    dydt = -y + x*y
11    return [dxdt, dydt]
12
13 # Parametros
14 alpha = 1.0
15 beta = 0.5
16
17 # Condicao inicial
```

Exemplo: Modelo Lotka-Volterra

Sistema Predador-Presa

$$\begin{aligned}\frac{dx}{dt} &= \alpha x - \beta xy \quad (\text{presas}) \\ \frac{dy}{dt} &= \delta xy - \gamma y \quad (\text{predadores})\end{aligned}$$

Parâmetros:

- α : taxa de crescimento das presas
- β : taxa de predação
- δ : eficiência de conversão
- γ : taxa de morte dos predadores

Ponto de Equilíbrio:

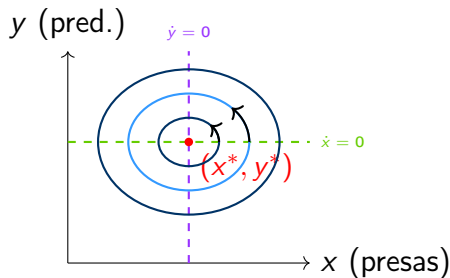
Retrato de Fase do Lotka-Volterra

Características:

- Órbitas fechadas (periódicas)
- Centro estável em (x^*, y^*)
- Amplitude depende da condição inicial
- Não há atrator: órbitas neutras
- Sistema conservativo

Nulclinas:

- $\dot{x} = 0$: $y = \alpha/\beta$
- $\dot{y} = 0$: $x = \gamma/\delta$



Interpretação:

Populações oscilam ciclicamente com defasagem temporal

Simulando Lotka-Volterra

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def lotka_volterra(X, t, alpha, beta, delta, gamma):
6     """Modelo predador-presa"""
7     x, y = X
8     dxdt = alpha*x - beta*x*y
9     dydt = delta*x*y - gamma*y
10    return [dxdt, dydt]
11
12 # Parametros
13 alpha, beta, delta, gamma = 1.0, 0.5, 0.5, 1.0
14
15 # Condições iniciais
16 X0 = [2.0, 1.0]
```

Pontos de Equilíbrio

Definição: Ponto de Equilíbrio (Ponto Fixo)

Um ponto $\mathbf{x}^*$ é um ponto de equilíbrio se:

$$\mathbf{f}(\mathbf{x}^*) = 0$$

Ou seja, todas as derivadas são zero: $\dot{x}_i = 0$ para todo i .

Interpretação Biológica

Pontos de equilíbrio representam:

- Estados estacionários do sistema
- Concentrações constantes ao longo do tempo
- Homeostase em sistemas biológicos
- Possíveis estados finais de longo prazo

Análise de Estabilidade Linear

Linearização em torno do Equilíbrio

Considere pequena perturbação $\xi(t) = x(t) - x^*$:

$$\frac{d\xi}{dt} = J(x^*)\xi$$

onde J é a matriz Jacobiana avaliada no equilíbrio.

Definição: Matriz Jacobiana

Matriz das derivadas parciais:

$$J_{ij} = \left. \frac{\partial f_i}{\partial x_j} \right|_{x^*}$$

Para sistema 2D:

$$\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix}$$

Autovalores e Estabilidade

Importante!

A estabilidade do equilíbrio $\mathbf{x}^*$ é determinada pelos autovalores λ_i da matriz Jacobiana $\mathbf{J}(\mathbf{x}^*)$.

Critério de Estabilidade

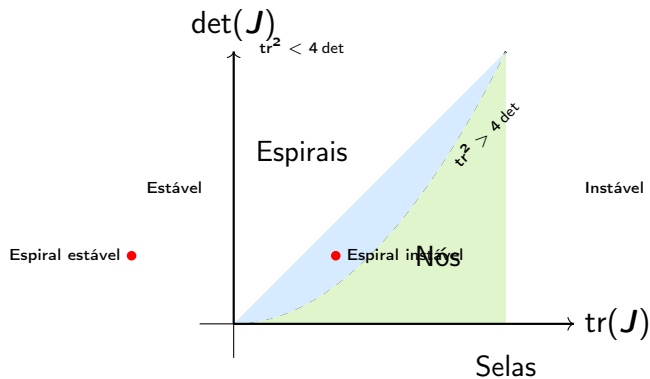
Para cada autovalor $\lambda = a + bi$:

- Se $\text{Re}(\lambda_i) < 0$ para todos $i \Rightarrow$ **estável**
- Se $\text{Re}(\lambda_i) > 0$ para algum $i \Rightarrow$ **instável**
- Se $\text{Re}(\lambda_i) = 0$ para algum $i \Rightarrow$ **marginal** (análise não-linear necessária)

Equação Característica (2D):

$$\det(\mathbf{J} - \lambda \mathbf{I}) = 0 \quad \Rightarrow \quad \lambda^2 - \text{tr}(\mathbf{J})\lambda + \det(\mathbf{J}) = 0$$

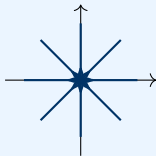
Classificação de Pontos Fixos em 2D



Tipos de Pontos Fixos

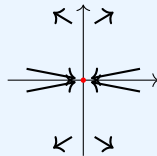
1. Nó Estável

- Ambos $\lambda_i < 0$ (reais)
- Convergência exponencial
- Sem oscilações



3. Ponto de Sela

- $\lambda_1 < 0 < \lambda_2$
- Instável (pelo menos uma direção diverge)



2. Nó Instável

- Ambos $\lambda_i > 0$ (reais)
- Divergência exponencial

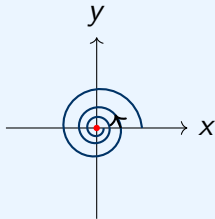
4. Centro

- $\lambda = \pm i\omega$ (imaginários puros)
- Órbitas fechadas

Espiraís: Estáveis vs. Instáveis

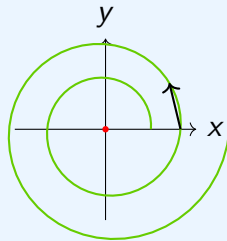
Espiral Estável

- $\lambda = a \pm bi$ com $a < 0$
- Convergência oscilatória
- Período: $T = 2\pi/|b|$
- Amortecimento: e^{at}



Espiral Instável

- $\lambda = a \pm bi$ com $a > 0$
- Divergência oscilatória
- Período: $T = 2\pi/|b|$
- Crescimento: e^{at}



Teorema de Estabilidade de Lyapunov

Definição: Estabilidade de Lyapunov

Um ponto de equilíbrio $\mathbf{x}^*$ é:

- **Estável:** se para todo $\epsilon > 0$ existe $\delta > 0$ tal que $\|\mathbf{x}(0) - \mathbf{x}^*\| < \delta$ implica $\|\mathbf{x}(t) - \mathbf{x}^*\| < \epsilon$ para todo $t \geq 0$
- **Assintoticamente estável:** se é estável e $\lim_{t \rightarrow \infty} \mathbf{x}(t) = \mathbf{x}^*$

Função de Lyapunov

Função escalar $V(\mathbf{x})$ tal que:

1. $V(\mathbf{x}^*) = 0$ e $V(\mathbf{x}) > 0$ para $\mathbf{x} \neq \mathbf{x}^*$
2. $\dot{V}(\mathbf{x}) \leq 0$ ao longo de trajetórias

Se existe tal V , então $\mathbf{x}^*$ é estável.

Se $\dot{V}(\mathbf{x}) < 0$ (estritamente), então é assintoticamente estável.

Calculando Jacobiano e Autovalores com Python

```
1 import numpy as np
2 from scipy.linalg import eig
3 import sympy as sp
4
5 # Definir variaveis simbolicas
6 x, y = sp.symbols('x y')
7
8 # Definir sistema
9 f1 = x*(1 - x - 0.5*y)
10 f2 = y*(-0.75 + 0.5*x)
11
12 # Calcular Jacobiano simbolicamente
13 J = sp.Matrix([[sp.diff(f1, x), sp.diff(f1, y)],
14                [sp.diff(f2, x), sp.diff(f2, y)]])
15
16 print("Jacobiano simbolico:")
17 print(J)
```

Exemplo: Análise de Estabilidade

Sistema:

$$\begin{aligned}\frac{dx}{dt} &= x(1 - x - 0.5y) \\ \frac{dy}{dt} &= y(-0.75 + 0.5x)\end{aligned}$$

Ponto de Equilíbrio: $(x^*, y^*) = (1.5, 1)$

Jacobiano:

$$J = \begin{bmatrix} 1 - 2x - 0.5y & -0.5x \\ 0.5y & -0.75 + 0.5x \end{bmatrix}$$

Em (1.5, 1):

$$J(1.5, 1) = \begin{bmatrix} -2.5 & -0.75 \\ 0.5 & 0 \end{bmatrix}$$

Autovalores: $\lambda = -1.25 \pm 0.61i$ (parte real negativa)

O que são Bifurcações?

Definição: Bifurcação

Uma bifurcação é uma mudança qualitativa na dinâmica de um sistema quando um parâmetro atravessa um valor crítico. Novos comportamentos emergem, equilíbrios aparecem/desaparecem ou mudam de estabilidade.

Importância Biológica

Bifurcações explicam:

- Transições entre estados celulares (diferenciação)
- Surgimento de oscilações (ritmos circadianos)
- Switches biestáveis (decisões celulares)
- Transições de fase em populações
- Limiar para patologias (onset de doenças)

Bifurcação Saddle-Node (Fold)

Equação Normal

$$\frac{dx}{dt} = \mu + x^2$$

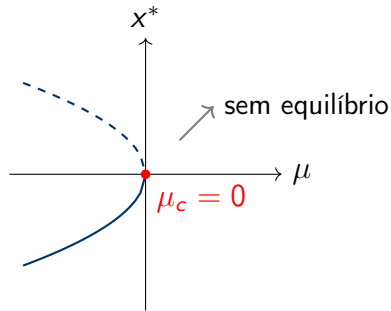
Análise:

- $\mu < 0$: dois equilíbrios
($x^* = \pm\sqrt{-\mu}$)
- $\mu = 0$: um equilíbrio
($x^* = 0$) - bifurcação!
- $\mu > 0$: nenhum equilíbrio

Interpretação:

- Colisão e aniquilação de equilíbrios

Diagrama de Bifurcação



Linha sólida: estável
Linha tracejada: instável

Bifurcação Transcrítica

Equação Normal

$$\frac{dx}{dt} = \mu x - x^2$$

Equilíbrios:

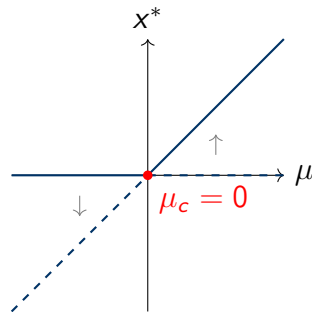
- $x^* = 0$ (sempre existe)
- $x^* = \mu$

Estabilidade:

- $\mu < 0$: $x = 0$ estável, $x = \mu$ instável
- $\mu > 0$: $x = 0$ instável, $x = \mu$ estável
- $\mu = 0$: troca de estabilidade!

Interpretação:

Diagrama de Bifurcação



Exemplo Biológico:

Transição de crescimento populacional

Bifurcação Pitchfork

Supercrítica (Suave)

$$\frac{dx}{dt} = \mu x - x^3$$

Equilíbrios:

- $x^* = 0$
- $x^* = \pm\sqrt{\mu}$ (se $\mu > 0$)

Estabilidade:

- $\mu < 0$: só $x = 0$ (estável)
- $\mu > 0$: $x = 0$ instável, $x = \pm\sqrt{\mu}$ estáveis

x^*
↑

Subcrítica (Abrupta)

$$\frac{dx}{dt} = \mu x + x^3$$

Equilíbrios:

- $x^* = 0$
- $x^* = \pm\sqrt{-\mu}$ (se $\mu < 0$)

Estabilidade:

- $\mu < 0$: $x = 0$ estável, $x = \pm\sqrt{-\mu}$ instáveis
- $\mu > 0$: $x = 0$ instável (divergência)

x^*
↑

Bifurcação de Hopf

Definição: Bifurcação de Hopf

Transição em que um equilíbrio estável perde estabilidade e dá origem a um ciclo limite periódico (oscilação).

Condições:

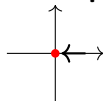
- Par de autovalores complexos conjugados: $\lambda = \alpha(\mu) \pm i\omega(\mu)$
- No ponto crítico μ_c : $\alpha(\mu_c) = 0$
- $\left. \frac{d\alpha}{d\mu} \right|_{\mu_c} > 0$

Tipos:

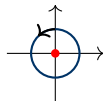
- **Supercrítica:** ciclo limite estável

Amplitude do ciclo:

Hopf Supercrítica

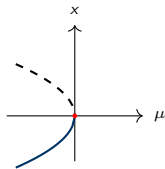


$\mu < \mu_c$
estável

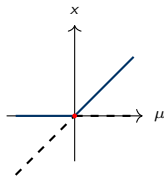


$\mu > \mu_c$
oscilação

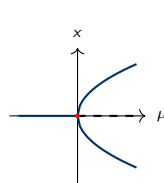
Diagramas de Bifurcação: Resumo Visual



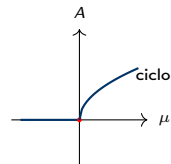
Saddle-node



Transcrítica



Pitchfork



Hopf

Gerando Diagramas de Bifurcação com Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4
5 def pitchfork(x, mu):
6     """Bifurcacao pitchfork supercritica"""
7     return mu*x - x**3
8
9 # Varrer parametro mu
10 mu_vals = np.linspace(-2, 2, 100)
11 equilibria = []
12
13 for mu in mu_vals:
14     # Encontrar todos os equilíbrios
15     eq_list = []
16     for x0 in [-2, 0, 2]: # tentativas iniciais
17         eq = fsolve(pitchfork, x0, args=(mu,))[0]
```

Soluções Periódicas

Definição: Solução Periódica

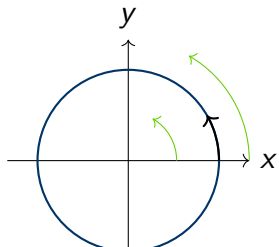
Uma solução $x(t)$ é periódica com período T se:

$$x(t + T) = x(t) \quad \forall t$$

No espaço de fase, corresponde a uma órbita fechada.

Tipos de Oscilações:

- **Linear:** oscilador harmônico (centro)
- **Não-linear:** ciclos limite
- **Forçada:** com input periódico



Ciclos Limite

Definição: Ciclo Limite

Uma órbita periódica isolada para a qual trajetórias próximas convergem (estável) ou divergem (instável). Diferente de centro, é estruturalmente estável sob perturbações.

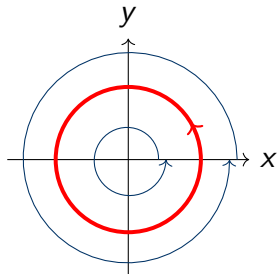
Propriedades:

- Órbita fechada isolada
- Amplitude e período fixos
- Atrator (se estável)
- Robusto a perturbações
- Não existe em sistemas lineares

Diferença de Centro:

- Centro: família de órbitas

Ciclo Limite Estável



Teorema de Poincaré-Bendixson

Teorema de Poincaré-Bendixson

Para sistemas 2D ($\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$), se:

1. Existe uma região limitada R positivamente invariante
2. R não contém pontos de equilíbrio
3. Uma trajetória $\mathbf{x}(t)$ permanece em R para $t \rightarrow \infty$

Então $\mathbf{x}(t)$ converge para um ciclo limite periódico.

Importante!

Este teorema garante a existência de oscilações em sistemas 2D! Não vale para dimensões maiores (onde caos é possível).

Aplicação:

Oscilador de Van der Pol

Equação de Van der Pol

$$\frac{d^2x}{dt^2} - \mu(1 - x^2)\frac{dx}{dt} + x = 0$$

onde $\mu > 0$ controla não-linearidade.

Sistema de primeira ordem:

$$\frac{dx}{dt} = y$$

$$\frac{dy}{dt} = \mu(1 - x^2)y - x$$

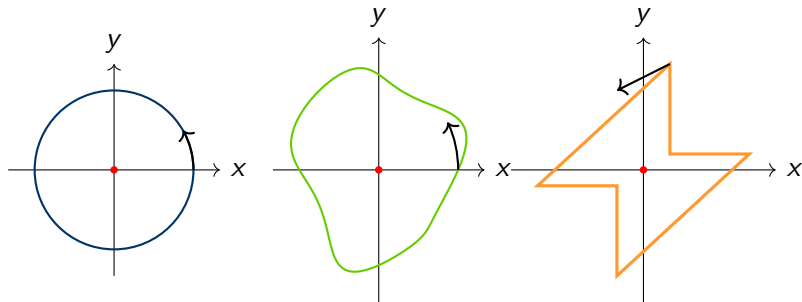
Características:

- μ pequeno: quase harmônico

Análise de Estabilidade:

$$J(0,0) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Retratos de Fase do Van der Pol



$\mu = 0.5$ (quase senoidal)

$\mu = 2$ (não-linear)

$\mu = 5$ (relaxação)

Importante!

Van der Pol é paradigmático para osciladores biológicos não-lineares!

Simulando Van der Pol

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def van_der_pol(X, t, mu):
6     """Oscilador de Van der Pol"""
7     x, y = X
8     dxdt = y
9     dydt = mu*(1 - x**2)*y - x
10    return [dxdt, dydt]
11
12 # Parametros
13 mu = 2.0
14
15 # Condicao inicial
16 X0 = [0.1, 0.1]
17
```

Osciladores Biológicos

Ritmos Circadianos

- Período $\sim 24\text{h}$
- Genes clock (Per, Cry, Bmal1)
- Feedback transcricional negativo
- Ciclo limite robusto

Oscilações Glicolíticas

- Período: $\sim 5\text{ min}$
- PFK (fosfofrutoquinase)
- Controle alostérico
- Coordenação metabólica

Oscilações de Cálcio

- Período: segundos a minutos
- IP3 e Ca^{2+} intracelular
- Sinalização celular
- Feedback positivo/negativo

Ciclo Celular

- Período: horas
- Ciclinas e CDKs
- Checkpoints
- Switches biestáveis + oscilador

Análise de Sensibilidade Local

Definição: Análise de Sensibilidade

Estudo de como mudanças nos parâmetros do modelo afetam as saídas (variáveis de estado, fluxos, propriedades emergentes).

Perguntas:

- Quais parâmetros são mais influentes?
- Quão confiáveis são as previsões?
- Onde concentrar esforços experimentais?
- Quais parâmetros são alvos terapêuticos?

Tipos:

- **Local:** derivadas parciais em ponto nominal
- **Global:** exploração de todo espaço paramétrico
- **Temporal:** dependência com tempo
- **Estrutural:** topologia do modelo

Coeficientes de Sensibilidade

Definição: Coeficiente de Sensibilidade Local

Derivada normalizada da saída y em relação ao parâmetro p :

$$S_{y,p} = \frac{\partial y}{\partial p} \cdot \frac{p}{y} = \frac{\partial \ln y}{\partial \ln p}$$

Mede a variação percentual em y quando p varia 1%.

Interpretação

- $|S_{y,p}| > 1$: alta sensibilidade (variação amplificada)
- $|S_{y,p}| < 1$: baixa sensibilidade (variação amortecida)
- $S_{y,p} > 0$: y aumenta com p
- $S_{y,p} < 0$: y diminui com p

Equações de Sensibilidade

Para sistema $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{p})$, definimos:

$$\mathbf{s}_i(t) = \frac{\partial \mathbf{x}(t)}{\partial p_i}$$

Equação de Sensibilidade

Derivando o sistema em relação a p_i :

$$\frac{d\mathbf{s}_i}{dt} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \mathbf{s}_i + \frac{\partial \mathbf{f}}{\partial p_i}$$

ou seja:

$$\frac{d\mathbf{s}_i}{dt} = \mathbf{J}(\mathbf{x}(t)) \mathbf{s}_i + \mathbf{g}_i(\mathbf{x}(t))$$

onde $\mathbf{J}$ é o Jacobiano e $\mathbf{g}_i = \partial \mathbf{f} / \partial p_i$.

Métodos Globais

Exploram todo o espaço paramétrico, não apenas um ponto:

1. Método de Morris (One-At-a-Time)

- Screening de parâmetros
- Elementary effects
- Identifica parâmetros importantes
- Computacionalmente eficiente
- Medidas: μ (efeito médio), σ (não-linearidade/interação)

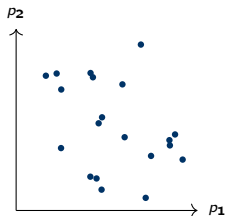
2. Análise de Variância (Sobol)

- Decomposição da variância
- Índices de Sobol
- S_i : efeito principal de p_i
- S_{ij} : interações entre p_i e p_j
- S_T^i : efeito total (inclui interações)

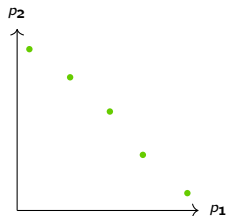
Exploração do Espaço Paramétrico

Técnicas de Amostragem

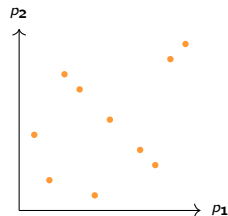
- **Monte Carlo:** amostragem aleatória
- **Latin Hypercube Sampling (LHS):** amostragem estratificada
- **Sobol sequences:** quasi-aleatória (baixa discrepância)



Monte Carlo



Latin Hypercube



Sobol Sequence

Exemplo: Sensibilidade de Modelo Cinético

Modelo: Reação enzimática simples

$$\frac{dx}{dt} = \frac{V_{max} \cdot S}{K_M + S} - k_{deg} \cdot x$$

Parâmetros: $V_{max} = 10$, $K_M = 5$, $k_{deg} = 0.5$, $S = 8$ (fixo)

Estado estacionário:

$$x^* = \frac{V_{max} \cdot S}{k_{deg}(K_M + S)} = \frac{10 \times 8}{0.5 \times 13} = 12.3$$

Sensibilidades em estado estacionário:

$$S_{x^*, V_{max}} = \frac{\partial x^*}{\partial V_{max}} \frac{V_{max}}{x^*} = 1$$

$$S_{x^*, k_{deg}} = \frac{\partial x^*}{\partial k_{deg}} \frac{k_{deg}}{x^*} = -1$$

Calculando Sensibilidade com Python

```
1 import numpy as np
2 from scipy.integrate import odeint
3
4 def modelo_sistema(x, t, Vmax, Km, kdeg, S):
5     """Modelo com parametros"""
6     dxdt = (Vmax * S) / (Km + S) - kdeg * x
7     return dxdt
8
9 # Parametros nominais
10 params_nom = {'Vmax': 10, 'Km': 5, 'kdeg': 0.5, 'S': 8}
11
12 # Calcular sensibilidade por perturbacao
13 def calc_sensibilidade(param_name, delta=0.01):
14     """Calcula coeficiente de sensibilidade"""
15     # Valor nominal
16     params_base = params_nom.copy()
17     t = np.linspace(0, 50, 500)
```

Análise Global com SALib

```
1 from SALib.sample import saltelli
2 from SALib.analyze import sobol
3 import numpy as np
4
5 # Definir problema
6 problem = {
7     'num_vars': 3,
8     'names': ['Vmax', 'Km', 'kdeg'],
9     'bounds': [[5, 15],      # Vmax: 50% variacao
10                [2.5, 7.5], # Km
11                [0.25, 0.75]] # kdeg
12 }
13
14 # Gerar amostras (metodo Saltelli para Sobol)
15 n_samples = 1024
16 param_values = saltelli.sample(problem, n_samples)
17
```

Exercícios - Parte 1: EDOs

Questão 1: Sistema Linear

Resolva analiticamente e numericamente (Python) o sistema:

$$\begin{aligned}\frac{dx}{dt} &= -x + 2y \\ \frac{dy}{dt} &= 2x - y\end{aligned}$$

com condições iniciais $(x_0, y_0) = (1, 0)$. Plote série temporal e retrato de fase.

Questão 2: Nulclinas

Para o sistema:

$$\frac{dx}{dt} = x(2 - x - y)$$

Exercícios - Parte 2: Estabilidade

Questão 3: Análise de Estabilidade

Analise a estabilidade de todos os equilíbrios do sistema:

$$\frac{dx}{dt} = y$$

$$\frac{dy}{dt} = -x - y + x^3$$

Calcule o Jacobiano, autovalores e classifique cada ponto fixo.

Questão 4: Lotka-Volterra Modificado

Considere o modelo com capacidade de carga:

$$\frac{dx}{dt} = x(1 - x/K - \alpha y)$$

Exercícios - Parte 3: Bifurcações

Questão 5: Diagrama de Bifurcação

Para $\dot{x} = \mu + x - x^3$, construa o diagrama de bifurcação completo:

1. Identifique o tipo de bifurcação
2. Determine valores críticos de μ
3. Plote ramos estáveis e instáveis
4. Gere o diagrama computacionalmente

Questão 6: Bifurcação de Hopf

Verifique numericamente a bifurcação de Hopf no sistema:

$$\begin{aligned}\frac{dx}{dt} &= \mu x - y - x(x^2 + y^2) \\ \frac{dy}{dt} &= x + \mu y - y(x^2 + y^2)\end{aligned}$$

Exercícios - Parte 4: Oscilações

Questão 7: Van der Pol

Simule o oscilador de Van der Pol para $\mu \in \{0.1, 1, 5\}$:

1. Compare amplitudes e formas de onda
2. Calcule período em cada caso
3. Plote retrato de fase para os três valores
4. Identifique regime de oscilações de relaxação

Questão 8: Modelo de Goodwin

Implemente o modelo de oscilador de Goodwin (3D):

$$\begin{aligned}\frac{dx}{dt} &= \frac{1}{1+z^n} - k_1 x \\ \frac{dy}{dt} &= x - k_2 y\end{aligned}$$

Exercícios - Parte 5: Sensibilidade

Questão 9: Sensibilidade Local

Para o sistema de síntese e degradação:

$$\frac{dx}{dt} = k_s - k_d x^n$$

com $k_s = 1$, $k_d = 0.1$, $n = 2$:

1. Calcule analiticamente S_{x^*, k_s} , S_{x^*, k_d} , $S_{x^*, n}$
2. Verifique numericamente por perturbação
3. Qual parâmetro tem maior impacto?

Projeto Prático

Escolha um modelo de rede regulatória (ex: toggle switch, repressilator) e:

1. Implemente o modelo em Python

Referências Principais

-  Strogatz S.H. (2015). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*, 2nd ed. Westview Press.
-  Murray J.D. (2002). *Mathematical Biology I: An Introduction*, 3rd ed. Springer.
-  Keener J., Sneyd J. (2009). *Mathematical Physiology*, 2nd ed. Springer.
-  Edelstein-Keshet L. (2005). *Mathematical Models in Biology*. SIAM.
-  Alon U. (2019). *An Introduction to Systems Biology: Design Principles of Biological Circuits*, 2nd ed. CRC Press.

Livros Avançados

- Kuznetsov Y.A. (2004). *Elements of Applied Bifurcation Theory*, 3rd ed. Springer.
- Wiggins S. (2003). *Introduction to Applied Nonlinear Dynamical Systems and Chaos*, 2nd ed. Springer.
- Ermentrout G.B., Terman D.H. (2010). *Mathematical Foundations of Neuroscience*. Springer.

Artigos Fundamentais

- Tyson J.J. et al. (2003). The dynamics of cell cycle regulation. *BioEssays* 25:1095-1109.
- Novák B., Tyson J.J. (2008). Design principles of biochemical oscillators. *Nat Rev Mol Cell Biol* 9:981-991.
- Ferrell J.E. (2012). Bistability, bifurcations, and Waddington's epigenetic

Software e Bibliotecas

- **Python:** `scipy.integrate` (`odeint`, `solve_ivp`), `sympy` (álgebra simbólica)
- **PyDSTool:** análise de sistemas dinâmicos, continuação, bifurcações
- **XPPAUT:** software clássico para EDOs e bifurcações
- **MATCONT:** continuação e bifurcações (MATLAB)
- **SALib:** análise de sensibilidade global (Python)

Tutoriais Online

- **SciPy Lecture Notes:** <https://scipy-lectures.org>
- **Dynamical Systems with Python:**
<https://github.com/cehberlin/dynamical-systems-python>
- **Strogatz's lectures:** YouTube (Cornell)

Obrigado!

Capítulo 4: The Mathematics of Biological Systems

Próximo Capítulo:
Estimação de Parâmetros e Ajuste de Modelos

Dúvidas? Perguntas?